

量化基础

目 录

1 引言：为什么需要量化	2
2 线性量化原理	2
2.1 直觉	2
2.2 形式化	2
2.3 回看	2
3 量化粒度	2
3.1 动机	2
3.2 三种粒度	3
4 对称量化与非对称量化	3
4.1 对称量化	3
4.2 非对称量化	3
5 RTN：最简单的基线	3
6 INT4 打包格式	4
7 反量化	4
8 NLL 评测	5
9 QuantizedWeight 数据结构	5
10 代码接口	6
11 实战：手动量化四个权重	6
12 本章你将学会	7
13 小结	7

1 引言：为什么需要量化

Gemma4-12B 有约 130 亿个参数。BF16 下每个参数占 2 字节，权重显存约 24 GiB。但 lab5 的 GPU 只有 10 GiB 显存 (H800 MIG 10G)，连模型权重都装不下，更不要说运行推理了。

量化 (quantization) 通过降低权重的数值精度来压缩显存。把 BF16 (16 bit) 降到 INT4 (4 bit)，每个参数从 2 字节缩到 0.5 字节，权重显存从 24 GiB 降到约 6 GiB，10 GiB 显存就够用了。

代价是精度损失：INT4 只有 16 个离散值，无法精确表示 BF16 的连续浮点数。如何选择量化参数、如何控制精度损失，是本章要讨论的核心问题。

2 线性量化原理

2.1 直觉

量化的本质是“四舍五入”：把连续的实数映射到有限的离散值。线性量化假设这种映射是线性的，即用一条直线把实数轴均匀切分成若干格，每格对应一个离散值。

2.2 形式化

给定一个实数 r (quantization 的“real”值)，我们把它映射到整数 q ：

$$q = \text{round}\left(\frac{r}{s} + z\right) \quad (2.1)$$

反过来，从整数 q 恢复近似实数 r ：

$$r = s \cdot (q - z) \quad (2.2)$$

其中 s 是**缩放因子** (scale)，决定一格的宽度； z 是**零点** (zero point)，决定实数轴上 0 对应哪个整数。

直觉 想象一把尺子： s 是刻度间距， z 是零刻度的位置。把实数对准刻度读数就是量化，把刻度读数乘以间距就是反量化。刻度越粗 (s 越大)，误差越大；刻度越细 (s 越小)，精度越高但表示范围越窄。

2.3 回看

线性量化用两个参数 s 和 z 描述一个仿射变换。量化时“四舍五入”到最近整数，反量化时用同一个变换倒推。误差来自舍入操作，大小取决于 s ： s 越大，每个整数代表的范围越宽，舍入误差越大。

3 量化粒度

3.1 动机

一个权重矩阵有上百万个参数，它们的数据范围可能差异很大。如果整个矩阵共用一组 s 和 z (per-tensor)，有些通道的值范围小，却被迫用很大的 s ，精度损失严重。把粒度变细，让每组通道有自己的 s 和 z ，可以更贴合数据分布。

3.2 三种粒度

粒度	共享范围	特点
per-tensor	整个张量	开销最小，精度损失最大
per-channel	每行/列	精度好，但 scales 数量多
per-group	每 G 列	折中方案，本实验采用

本实验采用 per-group 量化，group_size = 128：每 128 列共享一组 s 和 z 。这样每个权重矩阵有 $\frac{d_{in}}{G}$ 组参数，在精度和开销之间取得平衡。

per-group 是 per-channel 的推广：当 $G = 1$ 时退化为 per-element（每组一个参数），当 $G = d_{in}$ 时退化为 per-tensor。 $G = 128$ 是 INT4 量化中常用的经验值。

4 对称量化与非对称量化

4.1 对称量化

对称量化（symmetric quantization）令零点 $z = 0$ ，量化值关于 0 对称。INT4 对称量化的整数范围为 $[-8, 7]$ ，共 16 个值。

缩放因子 s 取权重组中绝对值最大的元素除以 7：

$$s = \frac{\max(|w|)}{7} \quad (4.1)$$

反量化时 $r = s \cdot q$ ，即 $w \approx s \cdot q$ 。

对称量化的好处是 $z = 0$ ，反量化不需要减零点，计算简单。坏处是如果权重的数据范围不对称（比如全是正数），会浪费一半的量化范围。

4.2 非对称量化

非对称量化（asymmetric quantization）允许 $z \neq 0$ ，量化值范围为 $[0, 15]$ 。缩放因子和零点的计算为：

$$s = \frac{\max(w) - \min(w)}{15}, \quad z = \text{round}\left(-\frac{\min(w)}{s}\right) \quad (4.2)$$

反量化时 $r = s \cdot (q - z)$ ，即 $w \approx s \cdot (q - z)$ 。

非对称量化充分利用了 $[0, 15]$ 的全部 16 个值，对于不对称的数据分布精度更好。但需要额外存储 z ，反量化时多一次减法。

直觉 对称量化像温度计的摄氏刻度，0 度居中，正负对称；非对称量化像考试百分制，0 是最低分，全部非负。如果数据天然不对称（如 ReLU 后的激活值），非对称更合适。

本实验中 GPTQ 量化默认使用对称量化， $z = 0$ ，整数范围 $[-8, 7]$ 。编码时把 $q + 8$ 映射到 $[0, 15]$ ，即 0 对应 -8，15 对应 7。

5 RTN：最简单的基线

RTN（Round to Nearest，最近舍入）是最简单的量化方法：对每个权重组独立计算 s 和 z ，然后直接舍入。它不考虑输入数据的分布，只看权重本身。

RTN 的流程:

1. 把权重矩阵按列分成若干组, 每组 G 列。
2. 对每组计算 $s = \frac{\max(|w|)}{7}$ (对称量化)。
3. 对每个权重 $q = \text{round}(\frac{w}{s})$, 截断到 $[-8, 7]$ 。

RTN 的优点是简单快速, 不需要校准数据。缺点是 INT4 下精度损失大: 它独立处理每个权重, 忽略了量化误差的累积效应。GPTQ 正是为了解决这个问题而提出的, 我们将在下一章详细讨论。

6 INT4 打包格式

INT4 量化后每个权重占 4 bit, 但内存按字节 (8 bit) 寻址。框架采用 `uint8_little_nibble` 格式把两个 INT4 值打包进一个 `uint8`:

- 低 **nibble** (低 4 bit) 存放偶数索引的权重
- 高 **nibble** (高 4 bit) 存放奇数索引的权重

例如, 两个量化值 $q_0 = 3$ (编码 $3 + 8 = 11$) 和 $q_1 = -4$ (编码 $-4 + 8 = 4$), 打包后:

$$\text{packed} = 4 \times 16 + 11 = 75 \quad (6.1)$$

即一个 `uint8` 值 75, 低 **nibble** 为 11 (q_0), 高 **nibble** 为 4 (q_1)。

```
def pack_int4(values: torch.Tensor) -> torch.Tensor:
    # values: int, range [-8, 7]
    # encode: value + 8 -> [0, 15]
    encoded = (values + 8).to(torch.uint8)
    # low nibble: even indices, high nibble: odd indices
    low = encoded[0::2]
    high = encoded[1::2]
    # pad if odd length
    if low.numel() > high.numel():
        high = torch.cat([high, torch.zeros(1, dtype=torch.uint8)])
    packed = low | (high << 4)
    return packed
```

`encoded` 把 $[-8, 7]$ 映射到 $[0, 15]$, 然后偶数索引放低 **nibble**, 奇数索引放高 **nibble**, 按位或合并成一个字节。

7 反量化

反量化把 INT4 值还原为浮点数。对称量化时:

$$w = s \cdot (q - 8) \quad (7.1)$$

这里 q 是编码后的值 $[0, 15]$, 减去 8 还原到 $[-8, 7]$, 再乘以缩放因子 s 。

非对称量化时:

$$w = s \cdot (q - z) \quad (7.2)$$

其中 z 是零点。

```
def dequantize_weight(quantized: QuantizedWeight, dtype: torch.dtype) ->
    torch.Tensor:
```

```

# unpack uint8 -> int4 values
low = qweight & 0x0F
high = (qweight >> 4) & 0x0F
values = torch.stack([low, high], dim=-1).flatten()
values = values[:numel].to(torch.int8) - 8 # [0,15] -> [-8,7]
# reshape to (out_features, in_features)
values = values.reshape(padded_shape)[: , :original_shape[1]]
# dequantize per group
if symmetric:
    weight = values * scales.repeat_interleave(group_size, dim=1)
else:
    weight = (values - zeros) * scales.repeat_interleave(group_size, dim=1)
return weight.to(dtype)

```

`qweight & 0x0F` 提取低 nibble, `>> 4` 后 `& 0x0F` 提取高 nibble。减去 8 还原到 $[-8, 7]$, 再按组乘以 `scales` 完成反量化。

8 NLL 评测

量化后我们需要衡量精度损失。**NLL** (Negative Log-Likelihood, 负对数似然) 是语言模型的标准指标:

$$\text{NLL} = -\frac{1}{L-1} \sum_{t=1}^{L-1} \log p(x_{t+1} | x_1, \dots, x_t) \quad (8.1)$$

NLL 衡量模型在测试集上预测下一个 token 的平均负对数概率。值越小, 模型预测越准确。

量化精度损失用 ΔNLL 衡量:

$$\Delta \text{NLL} = \text{NLL}_{\text{INT4}} - \text{NLL}_{\text{BF16}} \quad (8.2)$$

本实验要求 $\Delta \text{NLL} < 0.2$, 即量化后模型的预测能力不能有明显下降。

直觉 NLL 像考试的“扣分”: 模型每次预测下一个 token 时, 如果概率高 (猜对了), 扣分少; 概率低 (猜错了), 扣分多。量化后模型“变笨”了一点, 每次预测稍差, 扣分多一点。 $\Delta \text{NLL} < 0.2$ 就是要求“多扣的分”不超过 0.2。

9 QuantizedWeight 数据结构

框架用 `QuantizedWeight` 封装量化后的权重, 包含打包后的整数权重、缩放因子、零点以及元信息。

```

@dataclass
class QuantizedWeight:
    qweight: torch.Tensor # packed uint8, shape (out_features, in_features // 2)
    scales: torch.Tensor # (out_features, in_features // group_size)
    zeros: torch.Tensor | None # (out_features, in_features // group_size), None if symmetric

```

```

original_shape: tuple[int, int]
padded_shape: tuple[int, int]
bits: int = 4
group_size: int = 128
symmetric: bool = True
packing: str = "uint8_little_nibble"

```

`qweight` 是打包后的 uint8 张量，每字节含两个 INT4 值。`scales` 形状为 $(\text{out_features}, \text{in_features} // \text{group_size})$ ，每组一个缩放因子。`zeros` 在对称量化时为 `None`。`original_shape` 和 `padded_shape` 记录原始和填充后的形状（填充是为了对齐 `group_size`）。

10 代码接口

框架提供了 RTN 量化的完整接口：

```

def quantize_weight_rtn(
    weight: torch.Tensor,
    group_size: int,
    *,
    symmetric: bool = True,
    scale_dtype: torch.dtype = torch.float16,
) -> QuantizedWeight:
    ...

```

`weight` 形状为 $(\text{out_features}, \text{in_features})$ ，函数返回一个 `QuantizedWeight`。`group_size` 控制量化粒度，`symmetric` 选择对称或非对称，`scale_dtype` 指定缩放因子的存储精度。

11 实战：手动量化四个权重

我们取 4 个权重值，手动完成对称 INT4 量化的全过程。

例 权重： $w = [0.5, -1.2, 0.3, 2.1]$ ，`group_size = 4`（整组共享一个 `scale`）。

第 1 步：计算缩放因子

$$s = \frac{\max(|w|)}{7} = \frac{2.1}{7} = 0.3 \quad (11.1)$$

第 2 步：量化（舍入到整数）

$$q = \text{round}\left(\frac{w}{s}\right) = \text{round}([1.67, -4.0, 1.0, 7.0]) = [2, -4, 1, 7]$$

全部落在 $[-8, 7]$ 范围内，不需要截断。

第 3 步：编码到 $[0, 15]$

$$q_{\text{encoded}} = q + 8 = [10, 4, 9, 15]$$

第 4 步：打包为 uint8

低 nibble（偶数索引）： $q_0 = 10, q_2 = 9$

高 nibble（奇数索引）： $q_1 = 4, q_3 = 15$

$$\text{packed}_0 = 4 \times 16 + 10 = 74 \quad (11.2)$$

$$\text{packed}_1 = 15 \times 16 + 9 = 249 \quad (11.3)$$

第 5 步：反量化

$$w_{\text{dequant}} = s \cdot q = 0.3 \times [2, -4, 1, 7] = [0.6, -1.2, 0.3, 2.1]$$

第 6 步：计算误差

$$\text{error} = w_{\text{dequant}} - w = [0.1, 0, 0, 0]$$

最大绝对误差 0.1，出现在 $w_0 = 0.5$ 处。因为 $\frac{0.5}{0.3} = 1.67$ ，舍入到 2，反量化后 0.6，偏大了 0.1。其余三个权重恰好整除，无误差。

这个例子说明：缩放因子 s 由组内最大绝对值决定，小值权重的相对误差更大。这也是 per-group 量化的动机：缩小分组范围，让 s 更贴合每组的数值分布。

12 本章你将学会

1. 写出线性量化和反量化的公式，解释 s 和 z 的含义。
2. 区分 per-tensor、per-channel 和 per-group 三种粒度，说明本实验为何选择 `group_size = 128`。
3. 对称量化时计算缩放因子 $s = \frac{\max(|w|)}{7}$ ，非对称量化时计算 s 和 z 。
4. 描述 `uint8_little_nibble` 打包格式，说明低 nibble 和高 nibble 分别存放哪些索引。
5. 手动完成 4 个权重值的量化、编码、打包和反量化全过程。
6. 解释 NLL 的定义和 $\Delta \text{NLL} < 0.2$ 的含义。

13 小结

量化通过线性映射把高精度浮点权重压缩到低精度整数，核心是缩放因子 s 和零点 z 两个参数。本实验采用 per-group 对称 INT4 量化，每 128 列共享一组 s ，整数范围 $[-8, 7]$ ，打包为 `uint8_little_nibble` 格式。RTN 是最简单的量化基线，独立舍入每个权重，不考虑输入分布。NLL 衡量量化精度损失，要求 $\Delta \text{NLL} < 0.2$ 。下一章我们将学习 GPTQ 算法，它利用二阶信息补偿量化误差，在 INT4 下显著优于 RTN。

讲义基于 HPC Lab5 实验指导编写